PayLink database design

Scope and assumptions

PayLink is a **USD-only simulation**. There are no real bank links, payment networks, withdrawals, fees, currency conversion, password recovery, or real account numbers. An account number is unique only within its bank. A bank routing number is globally unique in this model. A label is a property of the shared account, not a separate nickname per owner. All joint owners have equal spending permission. A transfer has exactly one source, one destination, and one initiating user. Only completed transfers are stored; a rejected transfer rolls back and is not represented as a completed or pending row.

The application schema has exactly five base relations, matching the proposal. `paylink_meta.migrations` records deployment versions and is not a sixth domain entity. `paylink` is private: it must not be added to the exposed Supabase Data API schemas.

Editable conceptual and logical model

Open `er-diagram.drawio` in diagrams.net using **File → Open From → Device**. Entities, attributes, labels, and connectors are native editable objects. Save the XML file to preserve editability; exporting a PNG alone loses it. `er-diagram.mmd` is the editable Mermaid alternative, and `.dot` is Graphviz source. `.svg` and `.png` are rendered reference copies.

Relationship	Cardinality	Implementation and participation
Bank → BankAccount	1 : 0..N	Each account has exactly one non-null bank FK. A bank can have no accounts.
User ↔ BankAccount	M : N	`account_holders(user_id, account_id)`, with a composite PK and two FKs. A user may initially own zero accounts.
BankAccount → AccountHolder	1 : 1..N intended	The account-opening function inserts account and first holder atomically. SQL FKs alone do **not** enforce an account having at least one holder; administrative SQL can violate that minimum.
User → Transfer	1 : 0..N	One non-null `initiated_by` FK per transfer; not a duplicate of account ownership.
BankAccount → Transfer (source)	1 : 0..N	One non-null source FK.
BankAccount → Transfer (destination)	1 : 0..N	One non-null destination FK; source must differ.

No redundant direct `user_id` is stored on `bank_accounts`. A joint account therefore remains one account with one balance. Summing after an owner join can multiply its balance; account-scoped queries deduplicate by account ID.

Data dictionary

Relation	Keys	Non-key attributes and domains
`users`	PK `id`; alternate keys `username`, `email`	UUID id; lowercase username `[a-z][a-z0-9_]{2,29}`; lowercase unique email; nullable unique E.164-style phone; full_name 1–80 chars; Argon2id password_hash; created_at timestamptz.
`banks`	PK `id`; alternate key `routing_number`	Nine-digit routing string; name text. A name is not assumed unique.
`bank_accounts`	PK `id`; alternate key `(bank_id, account_number)`	Non-null bank FK; 12-digit synthetic account number; checking/savings type; shared label 1–60 chars; bigint balance and opening balance in cents, both 0–900,000,000,000,000; active boolean; created_at.
`account_holders`	PK `(user_id, account_id)`; both FKs	joined_at depends on the complete ownership pair. No redundant user name, bank name, or balance.
`transfers`	PK `id`; alternate key `(initiated_by, idempotency_key)`	Three non-null FKs; integer amount 1–100,000,000 cents; memo ≤140 chars; UUID idempotency key; timestamptz. Source and destination differ.

Phone is optional. PostgreSQL's normal UNIQUE semantics permit multiple NULLs, so **phone is not an unconditional candidate key**. For present phone values, uniqueness permits exact lookup. This distinction avoids treating SQL NULL as an ordinary value in the normalization proof. The syntax checks for email/phone establish format, not real-world verification.

Amounts use integer cents to exclude floating-point rounding. The maximum balance is below JavaScript's exact-integer limit. All accounts are USD, so no account currency or exchange-rate FD is introduced. Money is formatted only at the presentation boundary.

Functional dependencies and candidate keys

Use U=user_id, A=account_id, B=bank_id, T=transfer_id, K=idempotency_key.

- Users: `U → username, email, phone, full_name, password_hash, created_at`; `username → U`; `email → U`. U, username, and email are candidate keys under the stated case-normalization rules. Non-null phone determines U only conditionally.

- Banks: `B → routing_number, name`; `routing_number → B`. B and routing_number are candidate keys.
- Accounts: `A → B, account_number, account_type, label, balance_cents, opening_balance_cents, active, created_at`; `(B, account_number) → A`. These are the candidate keys. An account number alone does not identify an account.
- Holders: `(U,A) → joined_at`. Neither U nor A alone identifies an ownership row; there is no partial dependency.
- Transfers: `T → initiated_by, source_account_id, destination_account_id, amount_cents, memo, idempotency_key, created_at`; `(initiated_by,K) → T`. The key is scoped to the initiator, not globally to K.

These are the intended dependencies. We do not infer extra FDs from a small seed sample, such as “bank name determines bank” or “one user always uses one bank.”

From an unnormalized ownership record to BCNF

Start with a hypothetical ownership record containing user attributes, account attributes, bank attributes, and a repeating collection of owners. Repeating owners violate 1NF. Flattening the collection creates the 1NF relation:

`OwnershipWide(U, username, email, full_name, A, account_number, account_type, balance, B, routing_number, bank_name, joined_at)`

Its key is `(U,A)`. User details depend only on U and account details only on A. This creates partial dependencies and update anomalies: changing a name requires changing many rows; deleting the last ownership row can erase bank facts; a bank cannot be inserted independently.

2NF: Extract Users using U and AccountFacts using A; keep AccountHolders(U,A,joined_at). Each non-key holder attribute now depends on the whole key. AccountFacts still contains the transitive dependency `A → B → routing_number, bank_name`.

3NF: Extract Banks(B,routing_number,name) and retain only B as an FK on Accounts. User/bank/account descriptions are not copied into Transfers. Each non-key attribute now depends on a key without an intervening non-key determinant inside the relation.

BCNF: Under the declared FD set, every nontrivial determinant in each final relation is a superkey. The alternate keys are explicit UNIQUE constraints. Thus all five relations satisfy BCNF (and therefore 3NF), under these assumptions. We do not claim 4NF or 5NF without a separate multivalued/join-dependency analysis.

Lossless decomposition

For each binary split, the intersection determines one side:

1. Split Users from OwnershipWide: the shared attribute U determines the entire Users relation.
2. Split AccountFacts from the remainder: the shared A determines AccountFacts, leaving `(U,A,joined_at)`.

3. Split Banks from AccountFacts: shared B determines the Banks relation.

Each split therefore meets the FD-based binary lossless-join criterion. Their composition is lossless for the ownership relation, subject to its stated dependencies. Transfers represents a **separate event fact**; it is not obtained by naturally joining every ownership row. Blindly joining all five tables can multiply events for joint owners and is not a valid universal-relation proof.

Dependency preservation

The user, bank, account, and ownership determinants remain inside their corresponding relations. Transfer identity and idempotency dependencies remain inside Transfers. PK/UNIQUE constraints enforce those FDs without joining tables. FKs enforce referential integrity but are inclusion dependencies, not a replacement for FD-based normalization.

“Source belongs to initiator,” “sufficient balance,” “both accounts active,” and “money is conserved” span rows/relations. They are **business invariants**, not ordinary FDs or consequences of BCNF. The controlled transfer transaction enforces them. Minimum one holder per account is enforced by the supported creation workflow, not a general SQL assertion.

Transaction design

`execute_transfer(actor, source, destination, amount, memo, key)`:

1. Validate required fields and amount/domain restrictions.
2. Acquire a transaction advisory lock for the actor/key pair. Hash collisions can cause harmless extra serialization, not duplicate transfers.
3. Find an existing transfer. Return it if every input matches; reject reuse with different details. The unique constraint is an additional database safeguard.
4. Verify source ownership from AccountHolders. Future ownership-revocation workflows must coordinate with this check; revocation is not implemented now.
5. Lock both account rows in ascending UUID order with `FOR UPDATE`.
6. Check that both exist, are active, and source balance covers the amount.
7. Debit, credit, and insert the transfer in one transaction. Return the completed row.

READ COMMITTED with explicit account locks serializes conflicting balance updates. A waiting transaction sees the updated balance before its check. A consistent order prevents the classic A→B / B→A deadlock. The service must return success only after the database connection context commits. Unrelated account pairs can proceed independently; same-key retries serialize even if payload account pairs differ. There is no claim of serializability for arbitrary future workflows.

The private backend role cannot directly UPDATE balances, DELETE history, or INSERT transfers. It can read application relations, register users, and invoke the controlled functions. The functions are SECURITY DEFINER, have fixed `pg_catalog,paylink` search paths, and are not executable by PUBLIC. The actor UUID comes from a verified backend session, **not** an untrusted browser field. The backend role remains a trusted identity boundary: possession of its DB credentials permits impersonating actors via the function, which is why they must never reach the frontend.

Balance reconciliation and recovery

For account a:

`current_balance(a) = opening_balance(a) + Σ incoming(a) − Σ outgoing(a)`.

The materialized balance speeds sufficiency checks. It is a cross-relation aggregate redundancy, not an additional FD that invalidates BCNF. The transaction keeps the aggregate consistent; the reconciliation query independently detects divergence. Opening balances record initial simulated funding. New accounts create an explicit simulated allocation; only transfers between existing accounts conserve the total. No physical cash or regulatory balance is implied.

PostgreSQL's normal WAL/commit machinery provides durability and recovery; the application does not implement its own recovery log. Tests demonstrate transaction rollback, **not** power-failure recovery, backup restoration, or Supabase point-in-time recovery. Those remain future validation tasks. Privileged administrators can modify data; this is not an immutable double-entry ledger.

Physical design and query processing

UUID PKs and UNIQUE constraints create B-tree indexes. Additional indexes cover reverse ownership lookup and source/destination histories ordered by `(account_id, created_at DESC, id DESC)`. The descending UUID tie-breaker makes equal timestamps deterministic. Indexing is selective: there is no extra index for every small text column.

`queries.sql` demonstrates joins, GROUP BY/HAVING, CTEs, UNION ALL, reconciliation, a windowed running balance, storage inspection, and EXPLAIN(ANALYZE,BUFFERS). The measured 20,000-row microbenchmark is in `artifacts/query-evidence.txt`. Its synthetic rows and index drop are rolled back. It isolates the outgoing-history query; it does not prove that the entire UI history query or hosted workload has the same plan or speed.